

Lab7 Database Connectivity

ส่วนที่ 1: Software Design & Principles Explanation

1.1 สถาปัตยกรรมและ GRASP Patterns

โปรเจกต์นี้พัฒนาด้วยรูปแบบสถาปัตยกรรม **Model-View-Controller (MVC)** ร่วมกับการแบ่งชั้นการทำงาน (Layered Architecture) ประกอบด้วย Entity, Repository, Service และ Controller เพื่อแยกหน้าที่ของแต่ละส่วนให้ชัดเจน ลดการพึ่งพาซึ่งกันและกัน และทำให้ระบบสามารถดูแลรักษาและพัฒนาต่อได้ง่าย

Controller Pattern

GameController ทำหน้าที่รับ HTTP Request จากผู้ใช้ แล้วประสานงานกับ GameService เพื่อดำเนินการตามคำขอ เช่น แสดงข้อมูล เพิ่มข้อมูล แก้ไข และลบข้อมูล โดย Controller ไม่จัดการฐานข้อมูลหรือ Business Logic โดยตรง

Information Expert

Game Entity เป็นผู้เก็บข้อมูลของเกม เช่น ชื่อเกม แนวเกม ราคา วันวางจำหน่าย และประเภทส่วนลด รวมถึงมีเมธอดสำหรับคำนวณราคาหลังหักส่วนลดและแสดงชื่อโปรโมชั่น เนื่องจากเป็นคลาสที่มีข้อมูลครบถ้วน จึงเหมาะสมกับการเป็นผู้รับผิดชอบข้อมูลเหล่านี้

High Cohesion

แต่ละคลาสรับผิดชอบงานเพียงหน้าที่เดียว เช่น

- Entity จัดเก็บข้อมูล
- Repository ติดต่อฐานข้อมูล
- Service จัดการ Business Logic
- Controller รับ Request และส่งข้อมูลไปยัง View
- Strategy จัดการการคำนวณส่วนลด

การแบ่งหน้าที่ลักษณะนี้ทำให้แต่ละคลาสมีความรับผิดชอบเฉพาะด้าน

Low Coupling

Controller ไม่ติดต่อฐานข้อมูลโดยตรง แต่เรียกใช้งานผ่าน Service ส่วน Service ติดต่อ Repository เท่านั้น ทำให้แต่ละ Layer ไม่ขึ้นต่อกันโดยตรง หากมีการเปลี่ยนแปลงภายใน Layer หนึ่ง จะส่งผลกระทบต่อ Layer อื่นน้อย

Indirection

Service ทำหน้าที่เป็นตัวกลางระหว่าง Controller กับ Repository และ DiscountContext ทำหน้าที่เป็นตัวกลางในการเลือก Strategy ที่เหมาะสมสำหรับการคำนวณส่วนลด ทำให้แต่ละส่วนไม่จำเป็นต้องรู้รายละเอียดการทำงานของอีกฝ่าย

1.2 High-Level SOLID Principles

Single Responsibility Principle (SRP)

แต่ละคลาสมีหน้าที่เพียงอย่างเดียว เช่น GameRepository ติดต่อฐานข้อมูล ส่วน GameService จัดการ Business Logic และ GameController รับ Request จากผู้ใช้

Open/Closed Principle (OCP)

ระบบสามารถเพิ่มโปรโมชันใหม่ได้โดยสร้างคลาสที่ implements DiscountStrategy เพิ่มเติม โดยไม่ต้องแก้ไขคลาสเดิม ส่งผลให้ระบบรองรับการขยายความสามารถได้ง่าย

Liskov Substitution Principle (LSP)

ทุกคลาสที่ implements DiscountStrategy สามารถนำมาใช้งานแทนกันได้ เช่น NoDiscountStrategy, StudentDiscountStrategy และ SeasonalSaleStrategy โดยไม่กระทบการทำงานของระบบ

Interface Segregation Principle (ISP)

DiscountStrategy กำหนดเมธอด calculatePrice() เพียงเมธอดเดียว ทำให้คลาสที่นำไปใช้งานต้องรับผิดชอบเฉพาะสิ่งที่จำเป็น

Dependency Inversion Principle (DIP)

Controller ไม่สร้าง GameService เอง แต่รับผ่าน Constructor Injection และ Service ก็รับ GameRepository ผ่าน Constructor Injection เช่นกัน ทำให้ระบบอาศัย Abstraction มากกว่าการผูกติดกับ Implementation

1.3 Strategy Pattern

ระบบใช้ Strategy Pattern ในการคำนวณส่วนลดของเกม โดยกำหนด Interface ชื่อ DiscountStrategy ซึ่งประกอบด้วยเมธอด calculatePrice()

คลาสที่ implements ได้แก่

- NoDiscountStrategy : ไม่ลดราคา
- StudentDiscountStrategy : ลดราคา 10%
- SeasonalSaleStrategy : ลดราคา 20%

DiscountContext ทำหน้าที่เลือก Strategy ตามค่า discountType ของเกม เช่น NONE, STUDENT หรือ SEASONAL จากนั้นเรียกใช้ calculatePrice() เพื่อคำนวณราคาสุทธิ

ข้อดีของ Strategy Pattern คือสามารถเพิ่มโปรโมชั่นใหม่ได้ง่าย เช่น MemberDiscountStrategy หรือ HolidayDiscountStrategy โดยไม่ต้องแก้ไขโค้ดเดิม ส่งผลให้ระบบสอดคล้องกับหลัก Open/Closed Principle

1.4 Layered Architecture

ระบบแบ่งการทำงานออกเป็นหลาย Layer ได้แก่ Controller, Service, Repository และ Database

- Controller รับ Request และส่งข้อมูลไปยัง View
- Service ประมวลผล Business Logic และเรียกใช้งาน Strategy Pattern
- Repository ติดต่อฐานข้อมูล PostgreSQL ผ่าน Spring Data JPA
- Database ทำหน้าที่จัดเก็บข้อมูลจริง

การแบ่ง Layer ช่วยลดการเชื่อมโยงระหว่างส่วนต่าง ๆ ของระบบ (Low Coupling) และทำให้แต่ละ Layer มีหน้าที่ชัดเจน (High Cohesion)

1.5 Execution Flow

1. ผู้ใช้เปิดหน้าเว็บผ่าน Browser
2. Browser ส่ง HTTP Request ไปยัง GameController
3. Controller เรียกใช้ GameService
4. GameService ตรวจสอบข้อมูลและคำนวณราคาสุทธิผ่าน DiscountContext และ DiscountStrategy
5. GameService ติดต่อ GameRepository
6. Repository ใช้ Spring Data JPA ติดต่อ PostgreSQL
7. PostgreSQL บันทึกหรือดึงข้อมูลกลับมา
8. Repository ส่งข้อมูลกลับไปยัง Service
9. Service ส่งข้อมูลกลับไปยัง Controller
10. Controller ส่งข้อมูลไปยัง Thymeleaf Template เพื่อสร้างหน้า HTML
11. Browser แสดงผลลัพธ์ให้ผู้ใช้งาน

ส่วนที่ 2: Code Implementation & Explanation

Entity (Game)

Game Entity ใช้สำหรับเก็บข้อมูลของเกมและ Mapping กับตารางในฐานข้อมูลผ่าน JPA Annotation เช่น @Entity, @Table และ @Id โดยประกอบด้วยข้อมูล เช่น id, title, genre, platform, rating, releaseDate, price และ discountType

Repository

GameRepository สืบทอดจาก JpaRepository ทำให้สามารถใช้งาน CRUD ได้ทันที เช่น findAll(), findById(), save() และ deleteById() โดยไม่ต้องเขียน SQL เอง

Strategy Package

ประกอบด้วย DiscountStrategy, NoDiscountStrategy, StudentDiscountStrategy, SeasonalSaleStrategy และ DiscountContext สำหรับคำนวณราคาสุทธิตามประเภทส่วนลด

Service

GameService เป็นศูนย์กลางของ Business Logic ทำหน้าที่จัดการข้อมูลเกม เรียกใช้ Repository และเรียกใช้งาน DiscountContext เพื่อคำนวณราคาสุทธิ

Controller

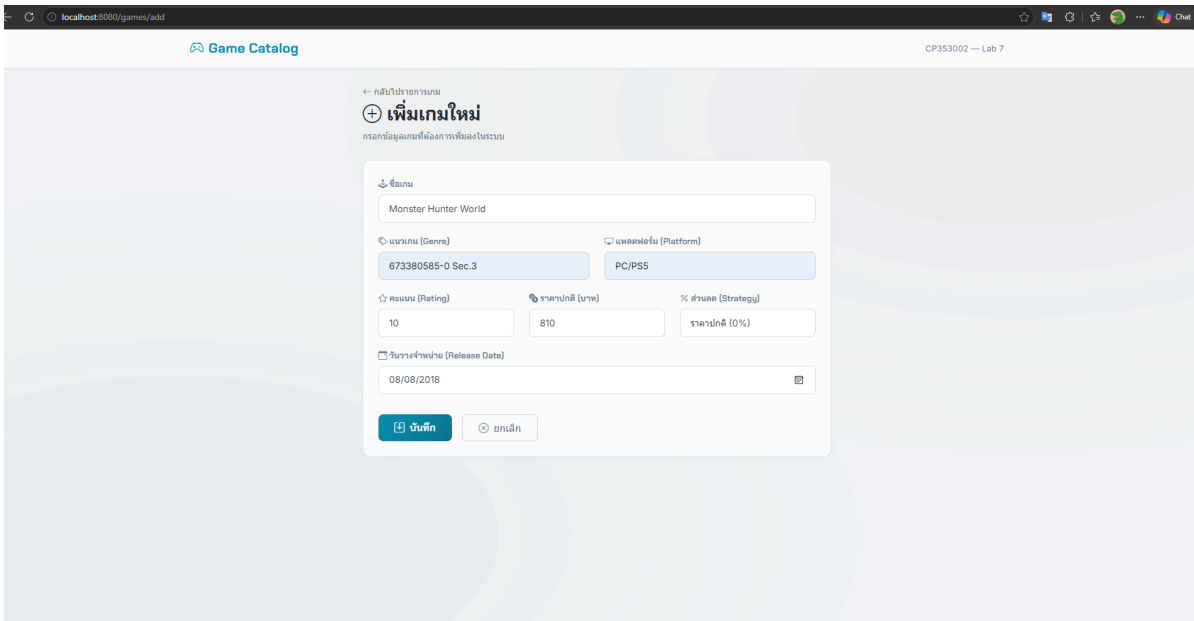
GameController รับ HTTP Request จากผู้ใช้ และเรียกใช้ GameService ผ่าน Constructor Injection จากนั้นส่งข้อมูลไปยัง Thymeleaf เพื่อแสดงผล

Dependency Injection (Constructor Injection)

ทุก Layer ใช้ Constructor Injection เพื่อรับ Object ที่ต้องใช้งาน เช่น GameController รับ GameService และ GameService รับ GameRepository วิธีนี้ช่วยลดการสร้าง Object ภายในคลาส ทำให้ระบบยืดหยุ่น ทดสอบได้ง่าย และสอดคล้องกับหลัก Dependency Inversion Principle

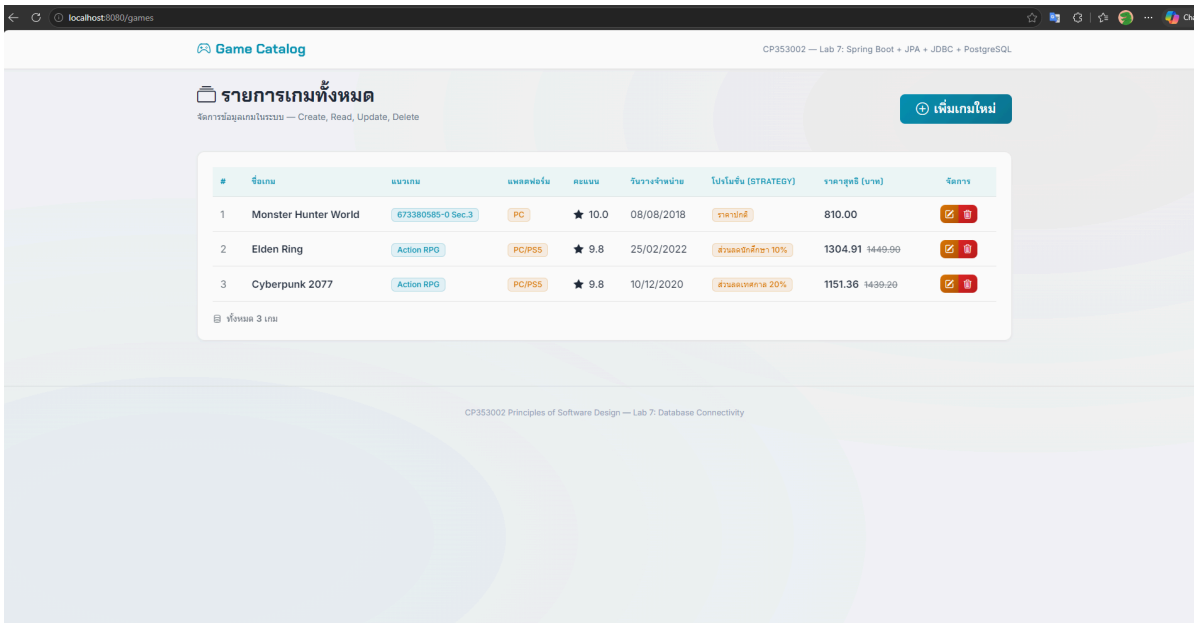
ส่วนที่ 3: Web Application & Database Screenshots

3.1 หน้าจอการเพิ่มเกมใหม่ (Create)



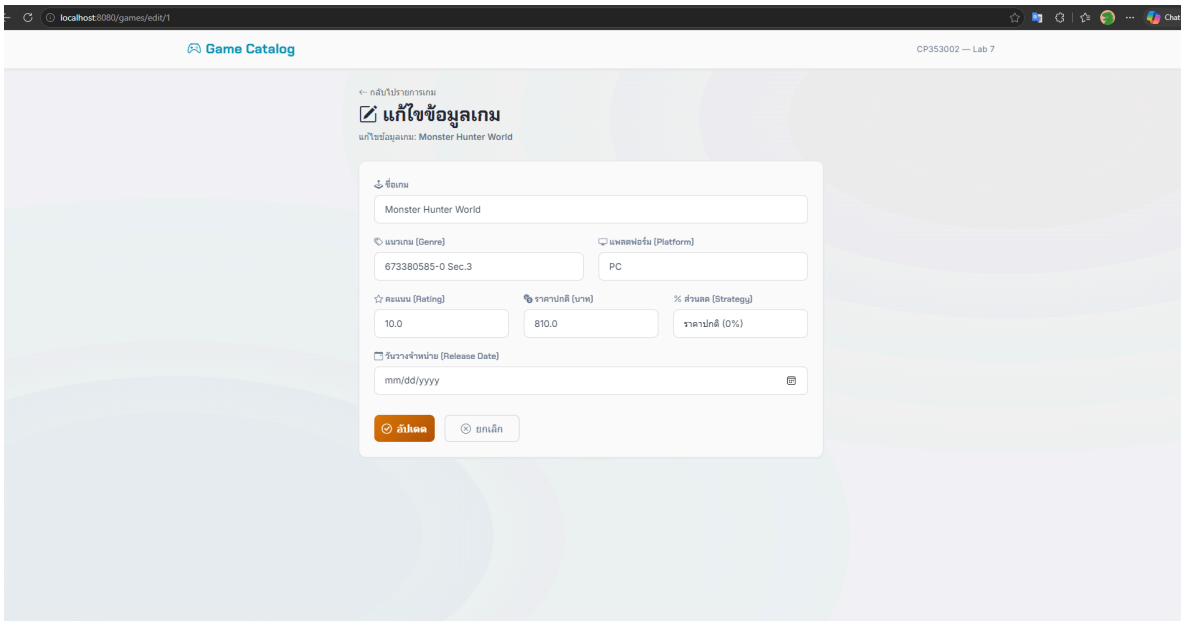
รูปที่ 3.1: หน้าฟอร์มเพิ่มเกมใหม่

3.2 หน้าจอแสดงรายการเกมทั้งหมด (Read)



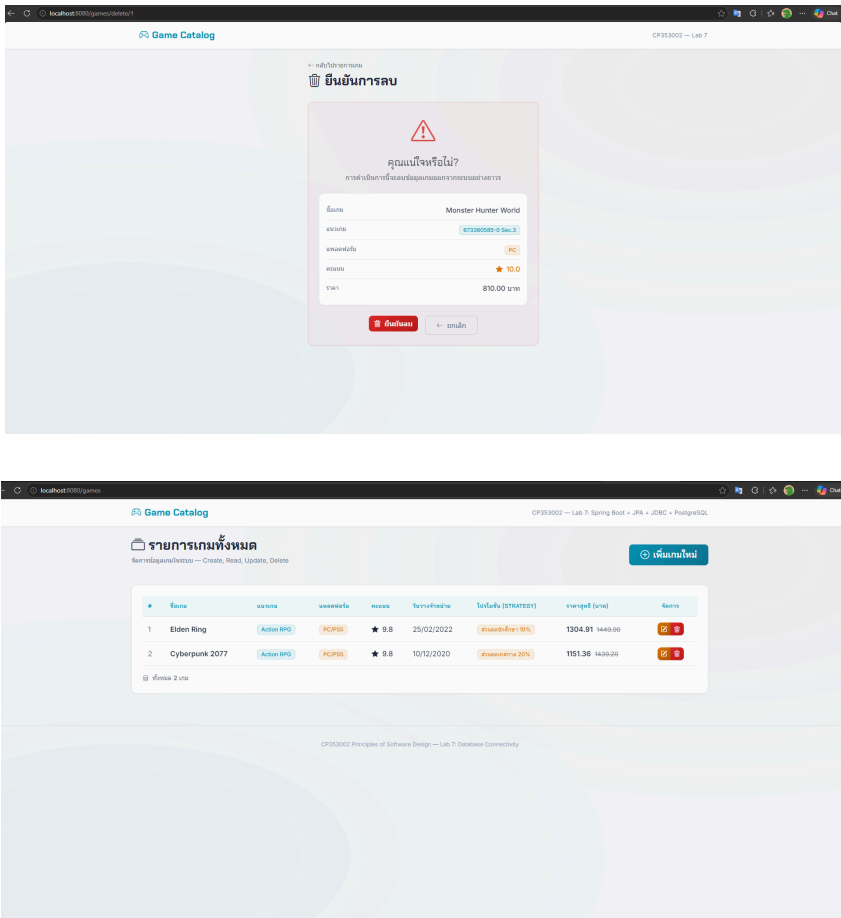
รูปที่ 3.2: หน้าแสดงรายการเกมทั้งหมด พร้อมแสดงราคาสุทธิที่คำนวณผ่าน Strategy Pattern

3.3 หน้าจอแก้ไขข้อมูลเกม (Update)



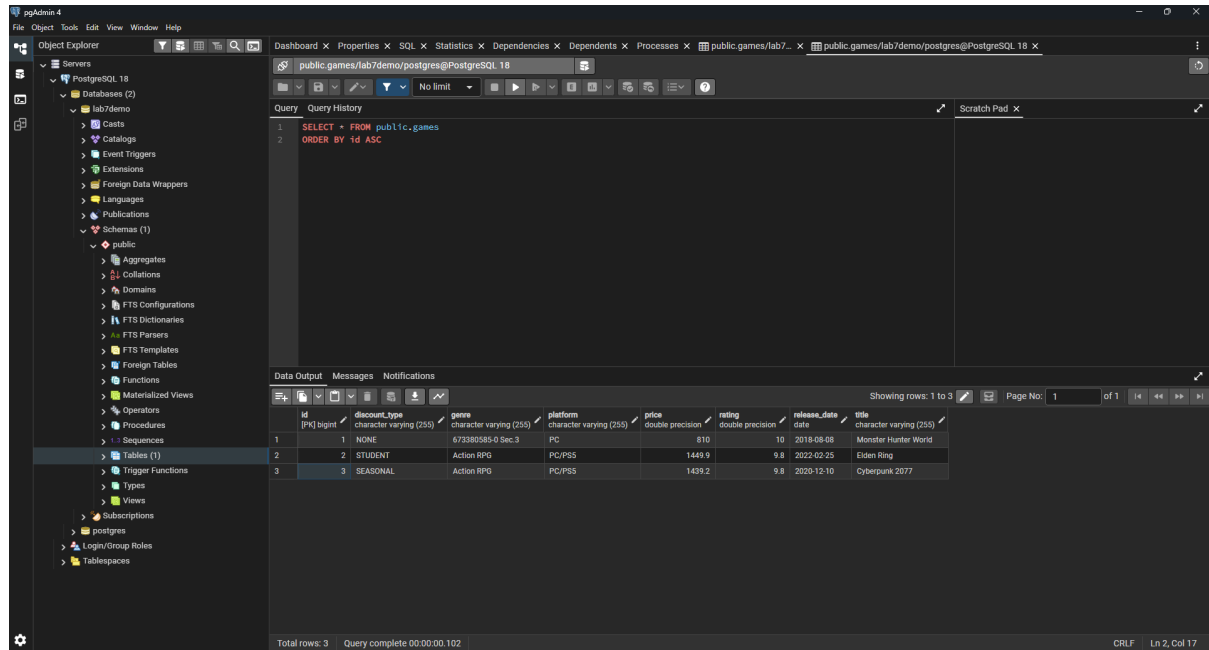
รูปที่ 3.3: หน้าฟอร์มแก้ไขข้อมูลเกม

3.4 หน้าจอยืนยันและการลบข้อมูลเกม (Delete)



รูปที่ 3.4: หน้าจอยืนยันการลบเกม และผลลัพธ์หลังทำการลบ

3.5 หน้าจอตรวจสอบข้อมูลใน PostgreSQL Database



The screenshot displays the pgAdmin 4 web interface. On the left, the 'Object Explorer' shows a tree structure of the database, with 'Tables (1)' selected under the 'public' schema. The main pane shows a SQL query: `SELECT * FROM public.games ORDER BY id ASC`. Below the query, the 'Data Output' tab displays the results of the query in a table format. The table has 8 columns: id, discount_type, game, platform, price, rating, release_date, and title. The results show 3 rows of data.

id	discount_type	game	platform	price	rating	release_date	title
1	1 NONE	673380585-0 Sec.3	PC		8.10	10 2018-08-08	Monster Hunter World
2	2 STUDENT	Action RPG	PC/PSS	1449.9	9.8	2022-02-25	Elden Ring
3	3 SEASONAL	Action RPG	PC/PSS	1439.2	9.8	2020-12-10	Cyberpunk 2077

At the bottom of the interface, it indicates 'Total rows: 3' and 'Query complete 00:00:00.102'.

รูปที่ 3.5: ข้อมูลตาราง games ที่ถูกจัดเก็บจริงในฐานข้อมูล PostgreSQL